

Threads in java

A thread is the direction or path that is taken while a program is being executed.

Thread is a lightweight sub process. It is the smallest independent unit of a program.

Multiple threads in Java

Multiple threads can be made to run parallel, this is done by extending the Thread class.

```
class class1Name extends Thread
```

```
{
```

```
    public void run()
```

```
    {
```

```
        //code block;
```

```
    }
```

```
}
```

```
class class2Name extends Thread
```

```
{
```

```
    public void run()
```

```
    {
```

```
        //code block;
```

```
    }
```

```
}
```

We should override the run() method of

the Thread class.

The OS scheduler schedules the execution of different threads.

start() method

We can use the start() method to call the run() method.

1. create a run() method.

2. create an object for the class.

```
className variableName = new className();
```

3. create a thread object by passing the classType variable.

```
Thread t1 = new Thread(variableName);
```

4. this will call the run() method.

```
t1.start();
```

Thread Priority

Every thread created in Java has some priority assigned to it, which ranges from 1 to 10.

5 : default priority

1 : minimum priority

10 : maximum priority

The statement

```
variableName.setPriority();
```

can be used to assign priority to a thread.

We can either set values (1-10) on thread class constants.

Thread.MAX_PRIORITY ~ 10

Thread.MIN_PRIORITY ~ 1

Thread.NORM_PRIORITY ~ 5

Sleep() method

The sleep() method is used to pause the execution for a specified duration.

```
try
{
    Thread.sleep(long millis);
}
catch (InterruptedException e)
{
    e.printStackTrace();
}
```

Runnable interface

We can also create threads by implementing the Runnable interface.

```
className implements Runnable
{
    //code block;
}
```


Race Condition

Race condition occurs when multiple threads read and write the same variable. they have access to some shared data and they try to change it at the same time. In such a scenario threads are "racing" each other to access/change the data.

In such scenarios, the program might produce anomalous results.

Thread - Safety

When a thread is already working on an object and preventing another thread on working on the same object, this process is called Thread - safety.

join() method

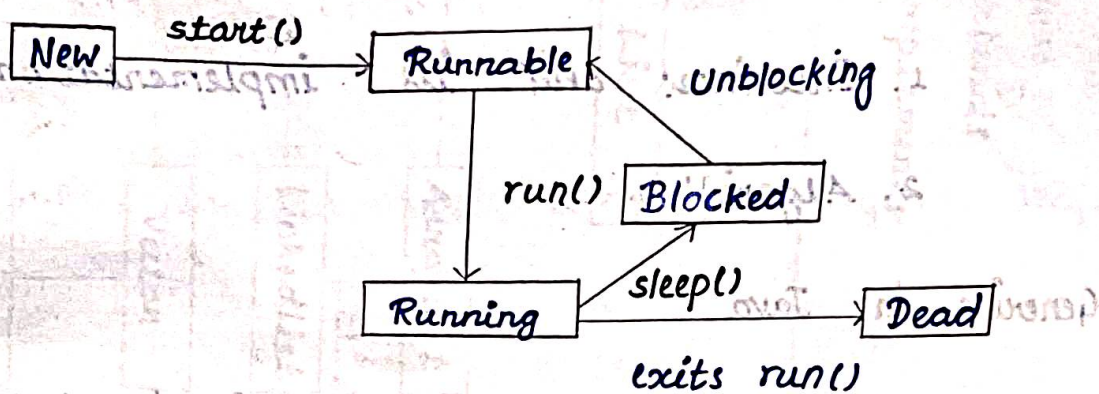
It will put the current thread on wait until the thread on which it is called is dead.

```
t1.join();  
t2.join();
```

"synchronized" keyword

It makes the resource accessible by only one thread at a time.

Thread States



New: A new thread is created but not working. A thread after creation and before invocation of start() method will be in new state.

Runnable: A thread after invocation of start() method will be in runnable state. A thread in runnable state will be available for thread scheduler.

Running: A thread in execution after thread scheduler select it, it will be in running state.

Blocked: A thread which is alive but not in runnable or running state will be in blocked state. It may be due to suspend(), sleep(), wait() methods or implicitly by JVM to perform I/O operations.

Dead: A thread after exiting from run() method will be in dead state. stop() method is used to forcefully kill a thread.